第2回：LLM APIの基礎

Pythonから gpt-6-luna を呼び出す

今日のゴールと流れ

APIへの入力と、返ってきた出力を説明できる。
LLMが前の会話を知らない理由と、履歴を送る方法を説明できる。

内容
前回の復習、モデルとAPI
HTTP、JSON、APIキー
完成版 `llm-call/` を動かす・読む
穴埋め：1回だけ呼ぶ、履歴をのせて呼ぶ
利用量、エラー、提出

API利用時は毎回、起動するターミナルで：
`export OPENAI_API_KEY=sk-********`
終了時は停止して `unset OPENAI_API_KEY` 。

今日の進め方

段階	すること
1	`sample-app/llm-call/` の完成版を動かす
2	コードの説明を聞く
3	`session02/exercise/` の穴あきコードを埋めて動かす
4	コミットしてプッシュする
5	GitHubのURLをフォームで提出する

穴あきコードの答えは `sample-app/llm-call/` です。

サービスの利用とAPIの利用

チャット画面では、人が入力し、画面に答えが表示されます。
APIでは、Pythonが入力を送り、Pythonが結果を受け取ります。

受け取った結果を画面に表示したり、他の処理へ渡したりできます。
チャット製品の契約とAPIの利用枠は、同じとは限りません。
講師の案内に従ってAPI利用環境を準備します。

HTTPとJSONの復習

要素	LLM APIでの意味
POST	入力を送り、回答の生成を依頼
JSONのリクエスト	モデル、指示、質問など
JSONのレスポンス	生成結果、状態、利用量など

Python SDKは、この通信を呼びやすくするライブラリです。

2種類のAPI

第3回以降は、APIが2か所に登場します。

API	呼ぶ側	受け取る側
自作の `/api/chat`	ブラウザ	自分のFastAPI
Responses API	自分のPython	外部のLLMサービス

今日は下の通信だけを扱います。Web画面はまだ使いません。

秘密情報の保管

APIキーを知る人は、利用権限を使える場合があります。

- 毎回の授業開始時に、起動するターミナルの環境変数へ設定する
- ファイル、Codespaces Secrets、シェルの設定ファイルに保存しない
- 終了時はサーバーを停止し、環境変数を解除してターミナルを閉じる
- キーの値を画面、ログ、提出物に貼らない

漏れた場合はキーを無効化・再発行します。Gitから消すだけでは不十分です。

毎回の授業で環境変数を設定

アプリを起動するターミナルで設定します。

```
export OPENAI_API_KEY=sk-********
```

同じターミナルから起動します。新しいターミナルでは再設定します。
終了時は `unset OPENAI_API_KEY` 。
キーはファイルに保存しません。 `.env` は読み込みません。

完成版を動かす

```
source .venv/bin/activate
cd sample-app/llm-call
python one_shot.py
```

「FastAPIを2文で説明して」と入力します。
回答と、入力・出力のトークン数が表示されて終わります。

キーの値を表示して確認する必要はありません。

今回のファイル

ファイル	役割
`one_shot.py`	質問を1つだけ送る
`chat.py`	質問と回答の履歴をためて、毎回まとめて送る

2つの違いは、`input` に入れるものだけです。
指示文（`INSTRUCTIONS`）は同じにしてあります。

読み順は `one_shot.py` の `main()`、`ask_once()`、`make_client()` です。

1問の処理を追う

質問：「FastAPIを2文で説明して」

順番	場所	起こること
1	`main()`	キーボードから質問を受け取る
2	`ask_once()`	質問と指示をSDKへ渡す
3	外部API	入力を受け取り、生成結果を返す
4	`ask_once()`	回答と利用量を辞書にまとめる
5	`main()`	辞書の内容を画面へ表示する

通信が発生するのは、SDKから外部APIへ依頼する部分です。

入口のコード

```
question = input('質問: ').strip()
if question == '' or len(question) > 500:
    print('質問は1〜500文字で入力してください。')
    return
result = ask_once(question)
print('回答:', result['answer'])
```

関数に質問を渡し、戻り値の辞書から必要な項目を取り出します。

`strip()` は文字列の前後の空白を除きます。

長さの確認は、APIを呼ぶ前に行います。

関数に渡す値と戻る値

```
question = 'FastAPIを2文で説明して'
result = ask_once(question)
answer = result['answer']
```

名前	型	意味
question	文字列	関数に渡す質問
result	辞書	回答と利用量のまとまり
answer	文字列	辞書から取り出した回答

辞書全体と、その中の回答文字列を区別します。

クライアントの作成

```
def make_client():
    if not os.getenv('OPENAI_API_KEY'):
        raise ValueError('…環境変数 OPENAI_API_KEY を設定してから実行してください。')
    return OpenAI(timeout=30.0, max_retries=0)
```

- `OPENAI_API_KEY` はSDKが環境変数から取得
- キーが無ければ、APIを呼ぶ前に分かりやすい文で止める
- `timeout` は応答待ちの時間、`max_retries=0` は自動再試行をしない設定

教材では、意図しない再送による追加呼び出しを避けます。

Responses APIの呼び出し

```
response = client.responses.create(
    model=MODEL,  # gpt-6-luna
    reasoning={'effort': 'none'},
    instructions=INSTRUCTIONS,
    input=question,
    max_output_tokens=600,
    store=False,
)
```

教材は短い応答の観察から始めるため、推論設定を `none` にします。

指定した項目の意味

項目	意味
model	呼び出すモデルのID
instructions	アプリが与える指示
input	今回の質問や参考情報
max_output_tokens	出力の上限。推論トークンも含む
store	Responses APIでの応答保存の指定

`store=False` だけで、あらゆるサービス側ログが無くなるわけではありません。
秘密情報は送信しません。

応答の取り出し

```
if response.status != 'completed' or response.output_text.strip() == '':
    raise ValueError('回答が完了しませんでした。質問を短くして試してください。')
return {
    'answer': response.output_text,
    'input_tokens': response.usage.input_tokens,
    'output_tokens': response.usage.output_tokens,
}
```

SDKの `output_text` で回答のテキストを取り出します。
途中で切れた回答や空の回答は、成功として返しません。

完了を確認する理由

応答の状態	教材の処理
completed で本文あり	回答と利用量を返す
completed 以外	ValueErrorで呼び出し元へ知らせる
本文が空白だけ	ValueErrorで呼び出し元へ知らせる

出力が途中で切れた場合も、正常な回答として表示しません。
質問や資料の長さを見直してから、再実行を判断します。

Pythonの例外

```
try:
    result = ask_once(question)
except ValueError as error:
    print(error)
    return
except APIError:
    print('API呼び出しに失敗しました。キー・モデル・利用枠を確認してください。')
    return
```

`try` で処理し、失敗したときは `except` へ進みます。
APIのエラー全文には内部の情報が含まれることがあるので、決まった文だけを表示します。

実習1：1回だけ呼ぶ

`session02/exercise/one_shot.py` の `ask_once()` を完成させます。

1. `TODO` を探す
2. `client.responses.create(...)` のコメントを外す
3. `raise NotImplementedError(...)` の行を消す
4. `cd session02/exercise` で `python one_shot.py` を実行する
5. 「FastAPIを2文で説明して」と入力する

埋める前に実行すると、`NotImplementedError` で止まります。

LLMは前の会話を覚えていない

`one_shot.py` を2回実行します。

実行	入力	期待する結果
1回目	私の名前は佐藤です。	あいさつが返る
2回目	私の名前は？	名前は分からない

2回目の `input` には、2回目の質問しか入っていません。
`store=False` なので、OpenAI側にも前の会話は残っていません。

履歴をのせて呼ぶ

`chat.py` は、質問と回答を `history` のリストにためます。

```
history = [
    {'role': 'user', 'content': '私の名前は佐藤です。'},
    {'role': 'assistant', 'content': 'こんにちは、佐藤さん。'},
    {'role': 'user', 'content': '私の名前は？'},
]
```

`role` は `user`（質問）か `assistant`（LLMの回答）です。
このリストを丸ごと `input` に入れて送ります。

one_shot.py と chat.py の違い

	one_shot.py	chat.py
input に入れるもの	今回の質問の文字列	これまでの質問と回答のリスト
前の会話	知らない	知っている
入力トークン数	ほぼ一定	質問を重ねるたびに増える

覚えているように見えるのは、アプリが履歴を毎回送っているからです。
chat.py は、履歴が長くなりすぎないよう質問を最大10回（ MAX_TURNS ）にしています。

chat.py の main() の流れ

```
for turn in range(1, MAX_TURNS + 1):
    question = input(f'質問{turn}: ').strip()
    if question == '':
        break
    # 1. 今回の質問を履歴に追加する
    # 2. 履歴をまとめて送る
    result = ask_with_history(history)
    # 3. LLMの回答も履歴に追加する
```

1と3の `history.append(...)` が、穴あきになっています。
3を忘れると、LLMは自分が何と答えたかを知らないまま次の質問を受けます。

実習2：履歴をのせて呼ぶ

`session02/exercise/chat.py` の `main()` を完成させます。

1. `TODO` を2か所探す（1の質問の追加、3の回答の追加）
2. 2か所の `history.append(...)` のコメントを外す
3. `raise NotImplementedError(...)` の行を消す
4. `python chat.py` を実行する
5. 「私の名前は佐藤です。」の次に「私の名前は？」と送る

何も入力せずにEnterを押すと終わります。

実習2の確認

確認	期待する結果
「私の名前は？」	佐藤と答えられる
表示される「送った履歴」	質問を重ねるたびに2件ずつ増える
入力トークン数	質問を重ねるたびに増える

3の追加を忘れた場合、「送った履歴」が1件ずつしか増えません。
その状態で「さっきの回答をもう一度」と頼むと、どうなるかも試せます。

実習3：指示と質問の比較

`one_shot.py` の `INSTRUCTIONS` だけを変え、同じ質問を送ります。

試行	指示の例
A	初学者向けに2文で説明する
B	箇条書きで3項目にまとめる
C	専門用語には短い説明を添える

質問は「FastAPIとは？」で固定します。
内容・形式・入力と出力のトークン数を見比べます。
同じ入力でも同じ結果を保証するとは限りません。

比較結果の読み取り

観点	観察する例
形式	指定した文数・項目数になったか
内容	同じ要点が含まれるか、条件が抜けていないか
用語	初学者が分からない用語に説明があるか
利用量	入力・出力のトークン数はどう変わったか

1回だけの差で断定せず、必要な例は同じ条件で再実行します。
追加の呼び出しにも利用量が発生するため、講師の予算内で行います。

利用料金の考え方

概算は、入力と出力のトークン数に、それぞれの単価を掛けます。

例：単価が100万トークン当たりの表示なら、

```
入力数 ÷ 1,000,000 × 入力単価
＋ 出力数 ÷ 1,000,000 × 出力単価
```

実際にはキャッシュ等の条件も関係します。
料金表と利用状況画面を講師と確認し、数値は開講時に更新します。

利用量の計算練習

説明用に、1問の入力を800、出力を200トークンとします。
同じ利用量の質問を10回送ると、入力8,000、出力2,000です。

100万トークン当たりの単価を、入力P円・出力Q円とすると：

概算料金 = 0.008 × P + 0.002 × Q 円

`chat.py` では履歴の分だけ入力が増えるので、10回目の入力は1回目より多くなります。
数値は計算練習用で、実測値や現在の料金ではありません。

よくあるエラー

状況	確認すること
キー設定の案内が出る	同じターミナルで `export` したか
認証エラー	キーの有効性。値そのものは共有しない
モデルを使えない	`gpt-6-luna` の利用権限・モデル名
利用制限	レート制限・利用枠・請求設定
待ち時間が長い	ネットワーク・入力の長さ

モデルを勝手に変更せず、講師に状況を伝えます。

失敗した場所の切り分け

観察した状態	まず確認すること
importで止まる	仮想環境と依存パッケージ
NotImplementedError	穴埋めと仮のraise
「回答が完了しませんでした」	入力の長さ
「API呼び出しに失敗しました」	キー・モデル・利用枠
名前を答えられない	3の `history.append` を外したか

相談時は、操作手順とエラーの種類を伝えます。キーは共有しません。

確認問題

1. ブラウザのJavaScriptにキーを書いてよい？
2. 出力を短くしても、入力トークンは料金に関係する？
3. `instructions` と `input` は何を分けている？
4. `chat.py` で前の会話を踏まえて答えられるのは、誰が何をしているから？

提出物と次回

`session02/exercise/` の2つのファイルをコミットし、プッシュします。

```
cd ../..  # リポジトリのルートへ戻る
git add session02/exercise
git commit -m "第2回: API呼び出しと履歴"
git push
```

GitHubで `session02/exercise` を開き、そのURLをフォームから提出します。
APIキーや、APIの生の例外全文は提出しません。

次回は、同じ処理をWeb画面から呼び出します。